

Geometric Data Analysis Project

# Vector and Heat Methods for Geometry Processing

Clément Marie

Ilan Aliouchouche

Karim Rochd

October 2025

## Selected Papers

- **The Vector Heat Method** — Nicholas Sharp, Yousuf Soliman, Keenan Crane.  
<https://www.cs.cmu.edu/~kmcrane/Projects/VectorHeatMethod/index.html>
- **The Heat Method for Distance Computation** — Keenan Crane, Clarisse Weischedel, Max Wardetzky.  
<https://www.cs.cmu.edu/~kmcrane/Projects/HeatMethod/index.html>

# Contents

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction &amp; Related Work</b>                       | <b>3</b>  |
| 1.1      | The Geodesic Problem . . . . .                               | 3         |
| 1.2      | Existing Methods . . . . .                                   | 3         |
| 1.3      | The “Heat” Intuition: Varadhan’s Formula . . . . .           | 3         |
| <b>2</b> | <b>Methodology A - The Scalar Heat Method</b>                | <b>3</b>  |
| 2.1      | The SHM Algorithm . . . . .                                  | 3         |
| 2.2      | Discretization & Robustness . . . . .                        | 4         |
| <b>3</b> | <b>Methodology B - The Vector Heat Method</b>                | <b>4</b>  |
| 3.1      | From Scalar to Vector . . . . .                              | 4         |
| 3.2      | The VHM Algorithm . . . . .                                  | 5         |
| 3.3      | Applications . . . . .                                       | 6         |
| 3.3.1    | The Logarithmic Map . . . . .                                | 6         |
| 3.3.2    | Geometric centers . . . . .                                  | 7         |
| <b>4</b> | <b>Experimental Evaluation &amp; Comparison</b>              | <b>8</b>  |
| 4.1      | Accuracy Analysis . . . . .                                  | 8         |
| 4.1.1    | Heat Method vs. Fast Marching vs. Exact Polyhedral . . . . . | 8         |
| 4.2      | Performance: The Power of Prefactorization . . . . .         | 8         |
| 4.3      | Robustness: Handling Difficult Meshes . . . . .              | 8         |
| 4.3.1    | Noise Resistance . . . . .                                   | 8         |
| 4.3.2    | Spurious Minima . . . . .                                    | 9         |
| 4.4      | Time Step Parameter Sensitivity . . . . .                    | 9         |
| 4.5      | Boundary Conditions . . . . .                                | 10        |
| <b>5</b> | <b>Discussion &amp; Limitations</b>                          | <b>11</b> |
| 5.1      | Role of the Time Step and Boundary Physics . . . . .         | 11        |
| 5.2      | Global and Method-Specific Limitations . . . . .             | 11        |
| 5.3      | Method Selection Guide . . . . .                             | 11        |
| <b>6</b> | <b>Conclusion</b>                                            | <b>11</b> |
| <b>7</b> | <b>References</b>                                            | <b>11</b> |

# 1 Introduction & Related Work

## 1.1 The Geodesic Problem

Computing distance on curved surfaces presents a fundamental challenge distinct from Euclidean geometry. While straight-line distance is trivial in flat space, the *geodesic distance*—the length of the shortest path along a curved surface—requires sophisticated approaches. A key distinction exists between *graph distance* and geodesic distance. Graph-based algorithms like Dijkstra’s constrain paths to mesh edges, systematically overestimating the true distance regardless of mesh refinement. Conversely, the multiple-source shortest path problem, fundamental to shape analysis and path planning, seeks the distance from every point on a domain to the closest point within a specific subset.

## 1.2 Existing Methods

Before the Heat Method, two main families of algorithms dominated: propagation-based methods and exact polyhedral methods.

**Fast Marching and Fast Sweeping.** These approaches formulate the problem via the non-linear *eikonal equation* ( $|\nabla\phi| = 1$ ), propagating distance information outwards from a source. While Fast Marching achieves  $O(n \log n)$  complexity, these methods suffer from significant practical limitations. They lack reusability, requiring distance computations to be restarted from scratch for each new source set. Furthermore, priority queues are inherently serial, hindering parallelization, while accurate implementation on triangulated surfaces often necessitates complex unfolding procedures or nonobtuse triangulations to avoid numerical errors.

**Exact Polyhedral Methods.** Algorithms by Mitchell, Mount, and Papadimitriou (MMP) and Surazhsky et al. compute exact paths that cut across triangle faces. Although mathematically precise, they are computationally expensive ( $O(n^2 \log n)$  or worst-case exponential depending on the variant) and implementation-heavy. They generally lack prefactorization capabilities, meaning every query starts anew, and are typically restricted to triangle meshes without direct generalization to point clouds.

Moreover, even “exact” polyhedral distances converge only quadratically to the true smooth geodesic distance, as the mesh itself is only an approximation of the underlying geometry.

## 1.3 The “Heat” Intuition: Varadhan’s Formula

The Heat Method is inspired by Varadhan’s formula (1967), which connects heat diffusion to geodesic distance. The intuition is that heat spreads over a surface such that, for a short time  $t$ , the amount of heat transferred between two points is determined by the geodesic distance between them. However, directly applying this formula fails numerically due to the exponential decay of the heat kernel. The Heat Method’s key insight is to use the heat kernel only to determine the *direction* of the geodesic (the gradient), discarding the unstable magnitude information and reconstructing the distance via the linear Poisson equation.

# 2 Methodology A - The Scalar Heat Method

The core contribution of this project relies on the *Heat Method* for geodesic distance computation. Unlike graph-based methods (e.g., Dijkstra) that are limited to edge paths, or PDE-based methods (e.g., Eikonal solvers) that are often non-linear and difficult to implement on curved domains, the Heat Method solves the geodesic distance problem using standard linear algebra.

The method is based on Varadhan’s formula, which establishes a link between the heat kernel  $k_t(x, y)$  and the geodesic distance  $d(x, y)$ :

$$\lim_{t \rightarrow 0} -\frac{\sqrt{4t}}{2} \ln k_t(x, y) = d(x, y) \quad (1)$$

While Varadhan’s formula theoretically requires  $t \rightarrow 0$ , numerical issues arise with vanishing time steps. The Heat Method circumvents this by utilizing the heat kernel only to determine the *direction* of the geodesic, rather than its magnitude.

## 2.1 The SHM Algorithm

The method proceeds in three distinct steps, transforming the problem into two linear Poisson-

like systems and one local normalization step.

**1. Integrate the Heat Flow.** The heat equation  $\dot{u} = \Delta u$  is first solved for a fixed source point  $x_0$  for a short time  $t$ . This describes how heat diffuses over the surface:

$$(M - tL_C)u = \delta_{x_0} \quad (2)$$

Here,  $\delta_{x_0}$  is the Kronecker delta (heat source), and a single backward Euler step is utilized for unconditional stability.

**2. Evaluate the Vector Field.** The heat kernel provides a smooth gradient field. The gradient of the heat distribution,  $\nabla u$ , points towards the heat source (steepest descent). Therefore, the vector field  $X$  pointing *away* from the source along geodesics is the normalized negative gradient:

$$X = -\frac{\nabla u}{\|\nabla u\|} \quad (3)$$

This step is performed locally per face. By normalizing the gradient, the incorrect magnitude provided by the heat kernel (which shrinks too fast) is discarded while the correct geodesic direction is preserved.

**3. Solve the Poisson Equation.** Finally, a scalar field  $\phi$  (the distance function) whose gradient best matches  $X$  is sought. The Poisson equation is solved:

$$L_C \phi = \nabla \cdot X \quad (4)$$

where  $\nabla \cdot X$  is the divergence of the normalized vector field. The solution  $\phi$  represents the geodesic distance from  $x_0$  to all vertices.

## 2.2 Discretization & Robustness

To implement this on a triangle mesh, the **Cotangent Laplacian**  $L_C$  is utilized. For an edge connecting vertices  $i$  and  $j$ , the weight is given by:

$$w_{ij} = \frac{1}{2}(\cot \alpha_{ij} + \cot \beta_{ij}) \quad (5)$$

where  $\alpha_{ij}$  and  $\beta_{ij}$  are the angles opposite the edge. This is combined with the lumped Mass matrix  $M$ , a diagonal matrix where  $M_{ii}$  is one-third of the area of all triangles incident to vertex  $i$ .

**Time Step Selection.** The accuracy of the method relies on the time parameter  $t = m \cdot h^2$  (where  $h$  is the average edge length). While  $m = 1$  is the theoretical standard, experiments on the coarse Bunny mesh required larger time scales (up to  $m = 64$ ). This suggests that for low-resolution geometry, a larger  $t$  is necessary to allow heat diffusion to bridge discretization artifacts and recover a smooth gradient.

**Boundary Conditions.** A critical implementation detail is the choice of boundary conditions. **Neumann boundary conditions** are applied, implying that no heat flows across the boundary ( $\nabla u \cdot \mathbf{n} = 0$ ). Using Dirichlet conditions (fixing boundary values to 0) artificially accelerates heat loss, warping the gradient field near edges and resulting in underestimated distances. The Cotangent Laplacian naturally implements Neumann conditions, ensuring distance estimates remain valid even on the rim of the mesh.

**Computational Efficiency.** The operators  $(M - tL_C)$  and  $L_C$  are symmetric positive-definite. These matrices are pre-factored using **Cholesky decomposition**. While the initial factorization is costly, subsequent distance computations for new source points are extremely fast (back-substitution only), making this method ideal for interactive applications compared to one-off Dijkstra searches.

## 3 Methodology B - The Vector Heat Method

### 3.1 From Scalar to Vector

While the Scalar Heat Method computes a distance field by diffusing a scalar function from a point source, the Vector Heat Method generalizes this idea to tangent vector fields. Instead of tracking how heat spreads across the surface, it tracks how an initial tangent vector at a source point is parallel transported along geodesics.

This shift from scalar to vector diffusion encodes not only how far points lie from the source, but also how directions evolve across curved geometry. The resulting vector field can be used to derive quantities such as the loga-

rhythmic map and geometric centers, discussed at the end of this section.

**Parallel Transport.** Let  $\gamma$  be the geodesic connecting points  $p$  and  $q$  on the surface. A tangent vector  $v_p \in T_p\mathcal{M}$  is transported along  $\gamma$  to obtain  $v_q \in T_q\mathcal{M}$  such that the intrinsic angle  $\varphi$  between the vector direction and the geodesic remains invariant. In flat space, the vector remains globally parallel; on curved geometry, curvature induces rotation, but the surface angle  $\varphi$  relative to  $\gamma$  is preserved.

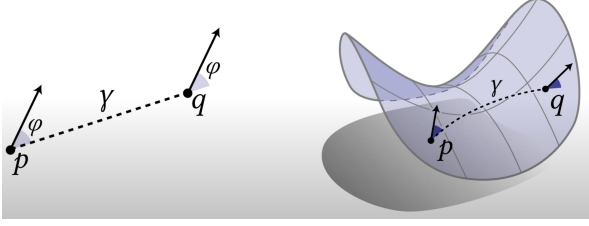


Figure 1: Parallel transport of a tangent vector along the geodesic  $\gamma$  from  $p$  to  $q$ . The intrinsic angle  $\varphi$  between the vector and the path is preserved on the surface. Figure taken from <https://www.cs.cmu.edu/~kmc Crane/Projects/VectorHeatMethod/index.html>.

**Discrete Formulation.** Each tangent vector is encoded as a complex number and collected into

$$U \in \mathbb{C}^n, \quad U_i \in T_{v_i}\mathcal{M} \simeq \mathbb{C}.$$

The discrete vector heat equation is

$$(M - tL^\nabla)U = U_0,$$

where  $M$  is the lumped mass matrix,  $U_0$  the complex Dirac field, and  $L^\nabla$  the connection Laplacian. Using complex numbers makes local rotations and parallel transport trivial (multiplication in  $\mathbb{C}$ ).

**Connection Laplacian.** To diffuse tangent directions rather than scalar values, the standard cotangent Laplacian is replaced by the connection Laplacian, which encodes how vectors rotate when moving across edges. In the complex representation, each edge  $(i, j)$  carries a rotation factor

$$r_{ij} \in \mathbb{C}, \quad |r_{ij}| = 1,$$

representing the change of tangent basis from  $i$  to  $j$ .

The operator is assembled as

$$L^\nabla \in \mathbb{R}^{n \times n},$$

with entries

$$(L^\nabla)_{ij} = \begin{cases} -r_{ij} w_{ij} & \text{if } (i, j) \text{ is an edge,} \\ \sum_{k \in N(i)} w_{ik} & \text{if } i = j, \\ 0 & \text{otherwise,} \end{cases}$$

where  $w_{ij}$  is the cotangent weight.

In matrix form:

$$L^\nabla = \begin{pmatrix} \sum_k w_{1k} & -r_{12}w_{12} & \cdots & -r_{1n}w_{1n} \\ -r_{21}w_{21} & \sum_k w_{2k} & \cdots & -r_{2n}w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ -r_{n1}w_{n1} & -r_{n2}w_{n2} & \cdots & \sum_k w_{nk} \end{pmatrix}$$

Here  $r_{ij}$  ensures that vector diffusion correctly transports directions across triangles, while magnitudes diffuse according to the same structure as the scalar Laplacian. This operator is the core ingredient distinguishing the Vector Heat Method from its scalar counterpart.

### 3.2 The VHM Algorithm

The method starts from an initial tangent vector field  $X$  at a source region and computes its transport across the mesh via four steps.

**1. Vector diffusion.**  $U_0 = X$  is set and the following is solved:

$$(M - tL^\nabla)U = U_0.$$

This transports directions across the surface in one linear solve.

**2. Magnitude diffusion.**  $u_0 = |X|$  (the magnitude of the initial vector field) is defined and the following is solved:

$$(M - tL_C)u = u_0.$$

This yields a smooth scalar field  $u$  encoding how far each vertex is from the source in vector-magnitude space.

**3. Dirac diffusion for normalisation.** Let  $\phi_0 = \mathbf{1}_{X \neq 0}$  (an indicator field of the source vertices) and solve

$$(M - t L_C) \phi = \phi_0.$$

$\phi$  gives a scalar field measuring geodesic reach from the non-zero source set.

**4. Scaling the result.** The final transported vector field is computed:

$$\bar{X} = \frac{u}{\phi} \cdot \frac{U}{|U|},$$

which combines the direction from  $U$  and the magnitude from  $u/\phi$ . This step is trivial (element-wise) and negligible in cost.

### 3.3 Applications

#### 3.3.1 The Logarithmic Map

The exponential map  $\exp_x : T_x \mathcal{M} \rightarrow \mathcal{M}$  follows the geodesic issued from  $x$  in the direction of a tangent vector  $v$ , and returns the point reached. The logarithmic map  $\log_x : \mathcal{M} \rightarrow T_x \mathcal{M}$  inverts this process: for any point  $y$ , it gives the tangent vector at  $x$  whose geodesic ends closest to  $y$ . Using the VHM, both the geodesic direction and the distance are recovered by diffusing vector fields, making the construction of  $\log_x$  linear and robust.

---

#### Algorithm 1 Logarithmic Map via the Vector Heat Method (VHM)

---

**Description.** The log map  $\log_x$  is approximated by polar coordinates  $(r, \varphi)$  around a base point  $x$ . Two smooth unit vector fields are built by vector heat diffusion: a horizontal field  $H$  (parallel transport of a reference direction at  $x$ ) and a radial field  $R$  (smoothed outward directions from  $x$ ). At each point, the angle between  $H$  and  $R$  gives the angular coordinate  $\varphi$ , while the radial coordinate  $r$  is recovered as a scalar whose gradient best matches  $R$ , by solving  $\Delta r = \nabla \cdot R$ . The log map is finally encoded as  $(u, v) = r e^{i\varphi}$ .

**Input:** Base point  $x \in \mathcal{M}$ ; horizontal field  $H_0 \in T_x \mathcal{M}$ ; radial field  $R_0 \in \mathbb{C}^n$ .

**Output:** Complex log-map  $(u, v) : \mathcal{M} \rightarrow \mathbb{C}$ .

- 1: Diffuse the horizontal field  $H_0$  using vector heat flow to obtain  $H$ ;
  - 2: Diffuse the radial field  $R_0$  to obtain  $R$ ;
  - 3: Compute the angular coordinate  $\varphi = \angle(H, R)$ ;
  - 4: Solve the Poisson equation  $\Delta r = \nabla \cdot R$  to recover  $r$ ;
  - 5: Return the logarithmic map  $(u, v) = r e^{i\varphi}$ .
-

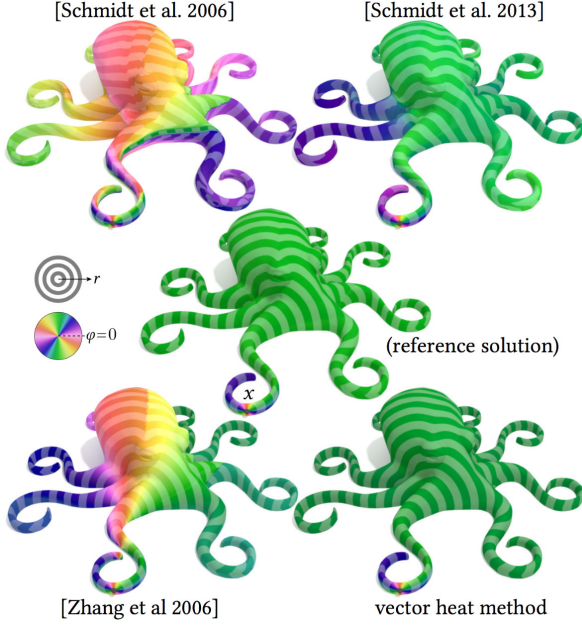


Figure 2: Comparison of log-maps between VHM and other methods. The VHM shows significantly sharper alignment to the reference solution, while older schemes propagate angular errors across the mesh. Because errors diffuse along the surface, even a small deviation at one region contaminates distant areas, leading to global drift.

### 3.3.2 Geometric centers

Now that logarithmic maps  $\log_x$  on a surface can be reliably evaluated, they can be used to define and compute geometric centers of a finite set of points  $y_1, \dots, y_n \in \mathcal{M}$ . A point  $x \in \mathcal{M}$  is called a geometric center if it minimizes the energy

$$E_p(x) = \frac{1}{2n} \sum_{i=1}^n d(x, y_i)^p,$$

where  $d$  denotes the geodesic distance on  $\mathcal{M}$ . For  $p = 2$ , minimizers are known as Karcher means; in Euclidean space this reduces to the usual arithmetic mean. For  $p = 1$ , minimizers are called geometric medians, which are typically more robust to outliers.

The log map naturally provides search directions for minimizing  $E_p$ : for a current estimate  $x$ , each vector  $\log_x(y_i)$  points from  $x$  toward a data point  $y_i$ . Averaging these vectors gives a direction that moves  $x$  toward all points at once. By repeatedly updating  $x$  using

the exponential map, a simple gradient-based algorithm that converges to a Karcher mean is obtained.

#### Algorithm 2 Karcher Mean via VHM

**Description.** The energy  $E_2(x) = \frac{1}{2n} \sum_i d(x, y_i)^2$  is minimized by iteratively averaging log vectors. At each iterate  $m^k$ , the VHM is used to compute  $\log_{m^k}(y_i)$  for all data points, these vectors are averaged to obtain an update direction, and movement along this direction is performed using the exponential map.

**Input:** Points  $y_1, \dots, y_n \in \mathcal{M}$ ; step size  $\tau > 0$ .

**Output:** Approximate Karcher mean  $m \in \mathcal{M}$ .

- 1: Pick a random initial guess  $m^0 \in \mathcal{M}$ ;
- 2: **repeat**
- 3:   Compute the log map at  $m^k$  (via VHM) to obtain  $\log_{m^k}(y_i)$  for all  $i$ ;
- 4:   Set the update vector  $v = \frac{1}{n} \sum_{i=1}^n \log_{m^k}(y_i)$ ;
- 5:   Update the estimate  $m^{k+1} = \exp_{m^k}(\tau v)$ ;
- 6: **until**  $\|v\|$  is below a tolerance
- 7: **return**  $m^{k+1}$ .

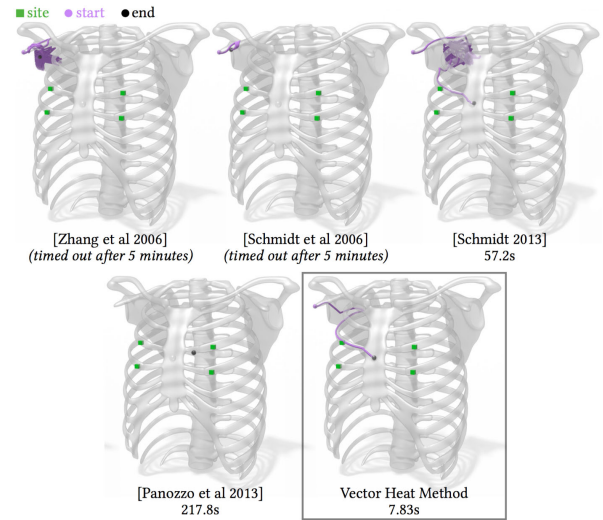


Figure 3: Karcher mean of four green points on a surface. Methods based on local log-map approximations may fail to converge or produce biased centers, whereas the VHM-based log map guides the iterates quickly to the correct symmetric location.

## 4 Experimental Evaluation & Comparison

The Scalar Heat Method was evaluated on standard **benchmark meshes** (Stanford Bunny, Armadillo, and others). This section presents a comprehensive comparison focusing on accuracy, performance, and robustness.

### 4.1 Accuracy Analysis

#### 4.1.1 Heat Method vs. Fast Marching vs. Exact Polyhedral

Table 1 presents a detailed comparison between the Scalar Heat Method, Fast Marching (Kimmel & Sethian), and Exact Polyhedral distance (MMP algorithm), as reported in Crane et al. (2013). Errors are measured relative to the exact polyhedral distance as a baseline.

**Key Observations.** Both methods appear to exhibit **linear convergence** ( $O(h)$ ) with respect to mesh refinement. Interestingly, even the exact polyhedral distance provides only **quadratic convergence** ( $O(h^2)$ ) to the true smooth geodesic distance. This is because the polyhedral surface itself is only an approximation of the underlying smooth geometry.

Fast Marching tends to achieve **smaller maximum error**, while the Heat Method often produces **better mean error**. The discrepancy in maximum error can be attributed to slight smoothing at sharp cusps (the cut locus), which is actually beneficial for many downstream applications.

### 4.2 Performance: The Power of Prefactorization

A key advantage of the Heat Method is that the linear systems in Steps I and III can be **prefactored**. The implementation uses sparse Cholesky factorization, which has sub-quadratic complexity and scales extremely well in practice.

**Amortized Cost Analysis.** For applications requiring distance from many different source points (shape analysis, remeshing, texture synthesis), the prefactorization strategy yields order-

of-magnitude speedups. After the one-time pre-computation:

- Each new distance query requires only back-substitution.
- The cost approaches  $O(n)$  for sparse systems from elliptic PDEs.
- Independent of source complexity (point, curve, or region).

As shown in Table 1, even for a single distance computation, the Heat Method outperforms Fast Marching; more importantly, updating distance for new source sets is consistently an order of magnitude faster (or more) than both Fast Marching and the exact algorithm.

In contrast, Fast Marching and Exact Polyhedral methods must recompute from scratch for each new source set, with no opportunity for reuse.

### 4.3 Robustness: Handling Difficult Meshes

Two factors contribute to the Heat Method’s robustness: (1) unconditionally stable backward Euler time discretization, and (2) elliptic rather than hyperbolic formulation (local averaging vs. sensitive wavefront propagation).

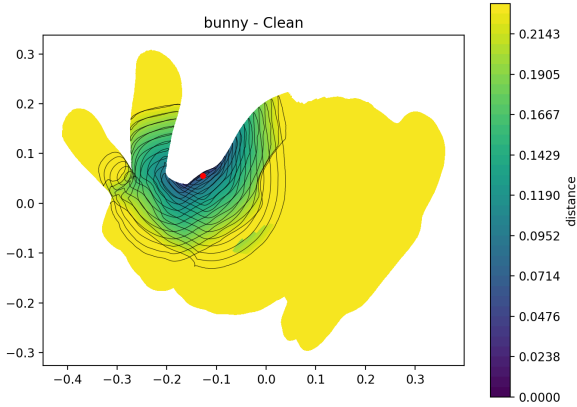
#### 4.3.1 Noise Resistance

Gaussian noise was applied to vertex positions to test stability.

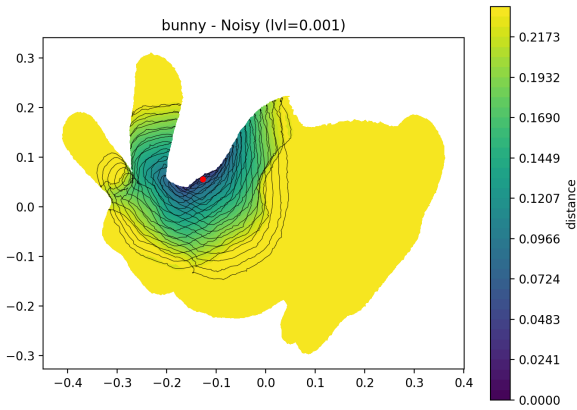


| Model     | Triangles | Heat Method |                      |       |        | Fast Marching |       |        | Exact   |
|-----------|-----------|-------------|----------------------|-------|--------|---------------|-------|--------|---------|
|           |           | Precompute  | Solve                | Max % | Mean % | Time          | Max % | Mean % | Time    |
| Bunny     | 28k       | 0.21s       | 0.01s (28 $\times$ ) | 3.22  | 1.12   | 0.28s         | 1.06  | 1.15   | 0.95s   |
| Isis      | 93k       | 0.73s       | 0.05s (21 $\times$ ) | 1.19  | 0.55   | 1.06s         | 0.60  | 0.76   | 5.61s   |
| Horse     | 96k       | 0.74s       | 0.05s (20 $\times$ ) | 1.18  | 0.42   | 1.00s         | 0.74  | 0.66   | 6.42s   |
| Kitten    | 106k      | 1.13s       | 0.06s (22 $\times$ ) | 0.78  | 0.43   | 1.29s         | 0.47  | 0.55   | 11.18s  |
| Bimba     | 149k      | 1.79s       | 0.09s (29 $\times$ ) | 1.92  | 0.73   | 2.62s         | 0.63  | 0.69   | 13.55s  |
| Aphrodite | 205k      | 2.66s       | 0.12s (47 $\times$ ) | 1.20  | 0.46   | 5.58s         | 0.58  | 0.59   | 25.74s  |
| Lion      | 353k      | 5.25s       | 0.24s (24 $\times$ ) | 1.92  | 0.84   | 10.92s        | 0.68  | 0.67   | 22.33s  |
| Ramses    | 1.6M      | 63.4s       | 1.45s (68 $\times$ ) | 0.49  | 0.24   | 98.11s        | 0.29  | 0.35   | 268.87s |

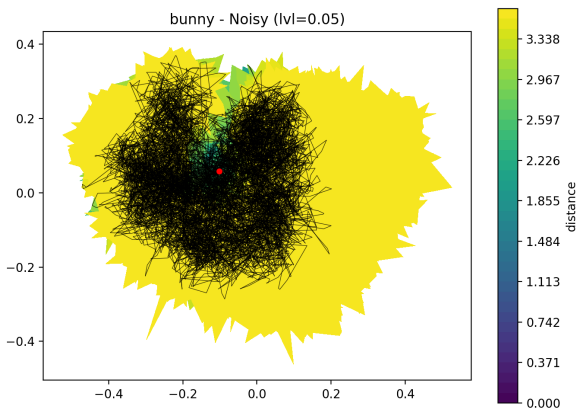
Table 1: Comparison of Heat Method, Fast Marching, and Exact Polyhedral distance (from Crane et al., 2013). Max error is percentage of mesh diameter; Mean is relative error per vertex. Speedup factors (in parentheses) indicate how many times faster each Heat Method solve is compared to Fast Marching.



(a) Clean Mesh (Reference)



(b) Low Noise ( $\sigma = 0.001$ ): 1.6% relative error



The method is stable with only 1.6% relative error at  $\sigma = 0.001$ . However, at high noise levels ( $\sigma = 0.02$ ), the relative error increases significantly. The noise adds high-frequency spikes that drastically increase the path length between points. This effect is similar to the Coastline Paradox: the more closely one follows the fine details of a jagged coastline, the longer it becomes.

**Comparison with Exact Methods.** While the Heat Method remains stable, exact polyhedral methods can become unreliable on poor-quality meshes. The Heat Method’s elliptic formulation acts as a natural regularizer, smoothing over local mesh artifacts while preserving global geometric structure.

#### 4.3.2 Spurious Minima

In finite element approximations, mesh quality affects solution quality. A common issue is “spurious minima”—local extrema that don’t exist in the true smooth solution. Empirically, the Heat Method produces **fewer spurious minima** than Fast Marching or biharmonic distance, due to regularization from the Hodge decomposition step (Step III).

#### 4.4 Time Step Parameter Sensitivity

A parameter sweep was performed on the Bunny mesh to analyze the effect of  $t = m \cdot h^2$ .

| Multiplier $m$ | Relative Error | Time (s) |
|----------------|----------------|----------|
| 0.25           | 55.8%          | 0.855    |
| 1.0            | 44.8%          | 1.156    |
| 4.0            | 25.6%          | 0.869    |
| 64.0           | <b>1.9%</b>    | 0.840    |

Table 2: Effect of time step on accuracy (Bunny mesh). Larger  $t$  helps on coarse meshes by allowing heat to diffuse across discretization artifacts.

### Why Larger $t$ Helps on Coarse Meshes.

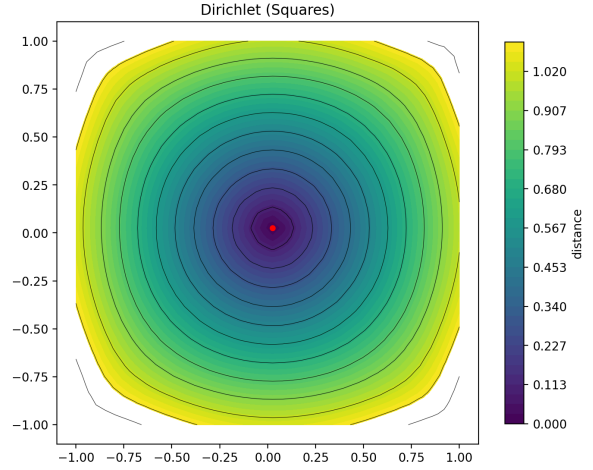
Theory suggests  $t \rightarrow 0$ , but on discrete meshes:

- Very small  $t$  traps heat within local triangulation artifacts.
- Moderate  $t$  allows heat to “bridge” mesh irregularities.
- Very large  $t$  over-smooths, ignoring surface curvature (cutting through geometry).

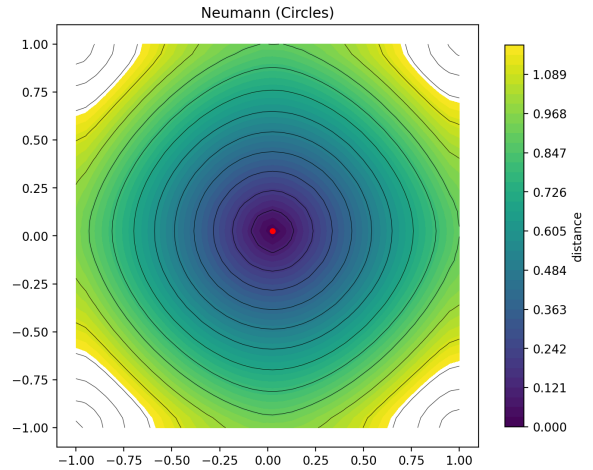
The default  $t = h^2$  works well for refined meshes; coarse meshes may benefit from larger values (up to  $t = 64h^2$ ).

## 4.5 Boundary Conditions

The necessity of Neumann boundary conditions was verified experimentally.



(a) Dirichlet ( $u|_{\partial\Omega} = 0$ )



(b) Neumann ( $\nabla u \cdot n = 0$ )

Figure 5: **Boundary Conditions.** (a) Dirichlet conditions force heat loss at the edge, causing isolines to warp inwards. (b) Neumann conditions (insulated) allow heat to flow along the edge, preserving the correct geodesic distance.

As seen in Figure 5, Dirichlet conditions cause the distance isolines to bend artificially near the boundary, whereas the Neumann implementation correctly treats the boundary as a “wall,” allowing accurate distance computation along the rim.

## 5 Discussion & Limitations

### 5.1 Role of the Time Step and Boundary Physics

Our experiments reveal that the time step  $t$  acts as a geometric regularizer rather than strictly following the theoretical limit  $t \rightarrow 0$ . Larger time steps allow heat to diffuse across discretization artifacts, effectively bridging irregularities on coarse meshes. However, excessive  $t$  values can smooth over genuine curvature, leading to underestimation. regarding boundaries, Neumann conditions ( $\nabla u \cdot \mathbf{n} = 0$ ) are strictly required to insulate the mesh, allowing heat to flow along the rim. In contrast, Dirichlet conditions force artificial heat loss, warping the gradient field and invalidating the distance computation near edges.

### 5.2 Global and Method-Specific Limitations

A universal challenge in geodesic computation is the *cut locus*—the set of points where the shortest path is not unique. At these locations, the distance function is continuous but non-smooth. The Heat Method handles this by averaging competing vector directions, yielding smooth gradients but losing exact locus geometry. Similarly, polyhedral methods only approximate the smooth cut locus, often diverging substantially from the true geometry.

Each approach carries distinct constraints. The Heat Method’s accuracy is sensitive to the time step choice and exhibits only linear convergence ( $O(h)$ ), making it less suitable for applications requiring strict polyhedral precision. The Vector Heat Method further requires intrinsic Delaunay triangulations to ensure numerical stability. Conversely, Exact Polyhedral methods are brittle, limited to triangle meshes, and lack the precomputation benefits that make heat-based methods efficient for repeated queries.

### 5.3 Method Selection Guide

Selection depends on the application’s specific needs. The Heat Method is optimal when speed and robustness are paramount, such as in shape analysis or texture synthesis involving noisy meshes

and multiple source queries. In contrast, Exact Polyhedral methods are preferable for tasks demanding maximum precision on high-quality meshes, where the computational cost of single-query processing is acceptable.

## 6 Conclusion

This project highlights the trade-off between the robust, approximate speed of heat diffusion and the brittle precision of exact polyhedral algorithms. The Heat Method’s shift from non-linear hyperbolic formulations to linear elliptic systems offers order-of-magnitude speedups via prefactorization, broad generality across geometric representations, and natural regularization against noise.

The Vector Heat Method extends these benefits to parallel transport, enabling robust computation of logarithmic maps and Karcher means. Future research may address current convergence limitations through higher-order discretizations, adaptive time-stepping based on local curvature, and GPU acceleration, further cementing the method’s role in modern geometry processing.

## 7 References

1. CRANE, K., WEISCHEDEL, C., & WARDETZKY, M. (2013). Geodesics in Heat: A New Approach to Computing Distance Based on Heat Flow. *ACM Transactions on Graphics*, 32(5).
2. SHARP, N., SOLIMAN, Y., & CRANE, K. (2019). The Vector Heat Method. *ACM Transactions on Graphics*, 38(3).
3. VARADHAN, S.R.S. (1967). On the behavior of the fundamental solution of the heat equation with variable coefficients. *Comm. Pure Appl. Math.*, 20(2), 431–455.
4. KIMMEL, R., & SETHIAN, J.A. (1998). Fast marching methods on triangulated domains. *Proc. Nat. Acad. Sci.*, 95, 8341–8435.
5. MITCHELL, J., MOUNT, D., & PAPADIMITRIOU, C. (1987). The discrete geodesic

- problem. *SIAM J. Comput.*, 16(4), 647–668.
6. SURAZHISKY, V., ET AL. (2005). Fast exact and approximate geodesics on meshes. *ACM Trans. Graph.*, 24(3), 553–560.
  7. BOBENKO, A., & SPRINGBORN, B. (2007). A discrete Laplace-Beltrami operator for simplicial surfaces. *Disc. Comp. Geom.*, 38(4).
  8. KNÖPPEL, F., CRANE, K., PINKALL, U., & SCHRÖDER, P. (2013). Globally optimal direction fields. *ACM Trans. Graph.*, 32(4).